Image Bundling Revisited: Atlasing Small Web Images under Modern Codecs and Protocols

Alexander Apartsin[1], Yehudit Aperstein[2]

[1]School of Computer Science, Faculty of Sciences, Holon Institute of Technology (HIT), Holon, Israel

[2]Intelligent Systems, Afeka Academic College of Engineering, Tel-Aviv, Israel

Technical Report · 2026

Abstract

Web pages load tens to hundreds of small images, and the median mobile page carries 13. Serving each one separately pays per-request overhead plus two byte costs that HTTP/2 multiplexing does not remove: a fixed per-file structural cost (roughly 600 bytes for a JPEG) and the loss of cross-image redundancy. We measure how much of these costs bundling many images into one atlas recovers, for the three codecs that carry most web images and decode in every browser: JPEG, PNG, and WebP.

At matched per-tile quality, atlasing small tiles saves up to 30% of bytes under JPEG and up to 19% under WebP, independent of protocol. The saving is set by the ratio of per-file overhead to content size, so it falls as tiles grow: for photographs it drops from about 30% at 56 pixels to a few percent at 224 pixels, where WebP turns clearly negative. Lossless formats lose under a naive grid but win 40–97% under codec-aware packing (vertical strips, shared palettes), so the question is how to pack, not whether. We distil the measurements into a coupling account (savings come from sharing fixed costs; losses from sharing adaptive state) and into an open-source tool that turns a directory of images into deployable bundles and matches an offline oracle on five unseen collections. A supporting browser study shows the byte savings also cut load time on HTTP/1.1 and HTTP/3.

1 Introduction

Product grids, icon sets, avatars, and decorative elements make small images the most numerous resource class on commercial pages. Bundling them into one image, the CSS sprite technique, was standard practice in the HTTP/1.1 era and fell out of favor when HTTP/2 multiplexing removed the per-connection request bottleneck. That reasoning addressed only the request-count cost. Two byte-level costs survive multiplexing untouched: every image file carries a fixed container overhead (headers, quantization tables, ISOBMFF boxes), and every file boundary prevents the codec from sharing entropy context, palettes, and predictors across images.

The study is deliberately scoped to the popular, universally-supported compressed formats: JPEG, PNG, and WebP together carry over 70% of images served on the web (HTTP Archive 2024: JPEG 32.4%, PNG 28.4%, WebP 12.0%), decode in every browser, and are where the practical savings live. The three formats price the per-file costs very differently: a JPEG file carries roughly 600 bytes of Huffman and quantization table definitions plus headers, a PNG a 67-byte structural floor plus per-scanline filter adaptation, and a WebP as little as 30 bytes of container around a heavily image-adapted VP8 payload. This report quantifies what bundling recovers, per codec, tile size, and image count, under a matched-quality protocol, and describes a testbed that measures the timing consequences under HTTP/1.1, HTTP/2, and HTTP/3.

The regimes this study measures map onto concrete page types. Product grids and image-search results serve 100–280-pixel photographic thumbnails by the dozens; recommendation strips, cart previews, and avatar rows serve 48–96-pixel photos; emoji and reaction pickers serve 24–64-pixel flat art by the hundreds, and are one place where sprite atlases remain in production use today (chat applications ship emoji sheets; video platforms ship hover-preview storyboards as frame mosaics). The tile-size axis of the experiments spans exactly this range, so each regime can read its expected saving off the measured curves.

The display side needs no special machinery: a bundled tile is shown by any of CSS `background-position` (universal), `object-view-box` (Chromium), canvas `drawImage` region blits, or SVG `viewBox` cropping. The browser decodes the atlas once and paints windows into it.

Contributions. This report contributes (i) a matched-quality sweep of image atlasing across the three universally-supported web codecs (JPEG, PNG, WebP) over tile size and image count, establishing that the byte saving is governed by the ratio of per-file structural cost to content bytes, and to our knowledge the first measurement of image-atlasing byte savings resolved by codec and tile size at matched perceptual quality (the one prior peer-reviewed sprite study [33] optimized PNG packing geometry and held the codec fixed); (ii) a supporting network study of 2,515 validated cold browser loads that separates the timing effect of bundling across HTTP/1.1, HTTP/2, and HTTP/3 under emulated network conditions; (iii) a set of construction techniques with measured effect, PNG vertical-strip packing, explicit and LZ-window duplicate exploitation, chunking for bounded cache invalidation, delta updates via compression dictionaries, and encoder-parameter tuning that flips WebP's photographic-atlas penalty to a gain; (iv) a coupling-spectrum account that unifies the results, savings arise from sharing fixed costs and losses from sharing adaptive state; and (v) an open-source construction heuristic that emits deployable bundles from a directory of images.

2 Related work

2.1 Resource bundling and image spriting

Combining many small resources into one is long-standing web practice, catalogued in the performance-engineering literature (Souders [35]) as spriting, concatenation, and DataURI inlining, all of which trade requests for cache granularity in the HTTP/1.1 era; CSS sprites [10] are the image-specific form. The regime they address is real and growing: measurement of page composition shows the modern page is dominated by many small, heterogeneous resources (Butkiewicz et al. [34]), and image requests are the largest class (HTTP Archive [8]). Whether bundling still pays under HTTP/2 multiplexing has been examined mainly for JavaScript and CSS: Khan Academy found unbundled JS slower than bundles under HTTP/2, attributing the gap to worse per-file compression [1], and practitioner analyses reach the same conclusion for concatenated assets [2, 11]. Modern build tools encode the trade-off directly as an inlining size threshold (webpack asset modules, Vite's asset limit). The closest academic work on the packaging question is Marx et al. [29], who test concatenation, embedding, and sharding under HTTP/2 and find the HTTP/1 packaging habits still often help; broader page-load studies show load time is governed by resource dependencies and compute, not bytes alone (WProf [26], How Speedy is SPDY? [27], Polaris [28]), so request-count reductions are only part of the story. The one peer-reviewed study of image spriting itself is Marszalkowski et al. [33], who formulate CSS-sprite construction as a geometric packing problem, model load time, and measure how a PNG sprite's aspect ratio affects its file size, holding the codec fixed at PNG and optimizing layout

area. Their axis is packing geometry; ours is codec and tile size at matched quality, which they do not vary. A CSS-Tricks case study reports a 223-icon sprite at roughly 10 KB versus 115 KB unbundled [2], but without protocol-level timing. We find no published measurement that quantifies image atlasing under HTTP/2 or HTTP/3, nor a codec-by-tile-size atlasing sweep at matched quality.

2.2 Protocol performance: HTTP/2 and HTTP/3

HTTP/2 multiplexes many requests over one connection [12] but suffers transport-level head-of-line blocking under loss; HTTP/3 over QUIC [13] removes it with per-stream recovery. Whether the newer protocol is faster in practice is contested and configuration-sensitive: de Saxcé et al. [36] found HTTP/2 not uniformly faster than HTTP/1.1, and rigorous QUIC evaluation shows performance swings widely with implementation and workload (Kakhki et al. [37]). Measurement studies characterize HTTP/2 adoption and page-load behavior [3] and server push [4], and both controlled benchmarks [5] and adoption studies [14] report HTTP/3 gaining most under packet loss while sitting near parity at zero loss. Domain sharding, the historical technique of spreading resources across hosts to widen HTTP/1.1 concurrency [15], is the opposite of bundling; recent measurement finds such HTTP/1-era habits persist even under HTTP/2 [30], which motivates our condition set. This body of work establishes that protocol-level outcomes for a given payload are as much a property of the deployment as of the standard, the context in which our own HTTP/3 concurrency observation (Section 5.5) should be read; none of it isolates a many-small-images payload against a bundled baseline.

2.3 Small-image coding and multi-image containers

The fixed per-file cost each codec carries is documented through minimal one-pixel files: JPEG XL 24 B, WebP 30 B, PNG 67 B, JPEG 155 B, AVIF 303 B [6, 7]. This overhead makes small images the worst case for heavyweight containers, and production pipelines act on it: Cloudinary declines AVIF for images below 5,000 pixels because the box overhead outweighs the coding gain [16]. Several formats already provide intermediate ways to share structure across many images without a full pixel atlas: JPEG's abbreviated format carries one table-specification datastream ahead of table-less scans [17]; animated WebP stores independently-coded frames in one container [19]; APNG and MNG [40] and the HEIF image-collection format [41] package multiple images in one resource. These occupy the low-coupling end of the spectrum this report maps, sharing container and tables but not the coding model. The JPEG [17], PNG [18], and WebP [19] format definitions specify the table, chunk, and container structures our measurements amortize; JPEG XL adds a modern architecture aimed partly at small images [38]. Matched-quality byte comparison across these formats is established for single images (Google's WebP study measures WebP against JPEG and PNG at equal SSIM [39]) and extended by peer-reviewed rate-distortion studies against newer codecs [31]; we carry the same matched-quality discipline to the atlas-versus-individual question. Perceptual image-quality assessment, on which our matched-quality protocol rests, is grounded in the structural similarity index [25]. We are not aware of prior work that measures how bundling recovers per-file overhead as a function of codec and tile size.

2.4 Texture atlases and sprite packing

Packing many images into one is standard in real-time graphics, where texture atlases [9, 20, 32] and skyline/MaxRects bin-packing [21] pursue a different objective: production atlas tools

(TexturePacker, engine sprite packers) and virtual-texturing systems minimize GPU state changes and packed area, and tune padding for mip-sampling and filtering correctness at tile boundaries rather than for encoded size. The optimization target is binding count and texture memory, not transmitted bytes, so this literature does not subsume a delivery-oriented study: our objective is encoded size and adaptive codec state under matched perceptual quality, which area-minimizing packers neither measure nor optimize.

2.5 Cache granularity and delta delivery

Bundling trades cache granularity for fewer requests, a tension delivery research has long addressed. HTTP delta encoding transmits only the difference between a cached and a current resource (RFC 3229 [42]) using generic differencing formats (VCDIFF, RFC 3284 [43]); shared-dictionary schemes reuse previously delivered bytes as a dictionary for later responses (SDCH [44]), the idea that shared-window and trained-dictionary compressors (Brotli [22], zstd [23]) generalize and that Compression Dictionary Transport [24] revives for the modern web. Standard HTTP caching semantics (RFC 9111 [45]) fix the granularity at the resource, which is exactly what bundling coarsens; we apply delta and dictionary delivery to the whole-bundle cache-invalidation problem (Section 6.2). HTTP Archive's Web Almanac [8] supplies the population statistics (median 13 images per mobile page, format share) that make the many-small-images regime the common case rather than an edge case.

3 The atlas serving model

3.1 Rendering a tile from an atlas in HTML

A bundled page references one image resource and displays each tile by cropping a window into it. Four standard mechanisms cover every deployment context. The classic and universal one positions the atlas behind a fixed-size element as a background:

```
<div class="tile" style="background-image:url(atlas.webp);
     background-position:-144px -216px"></div>
/* .tile has width:72px; height:72px */
```

The element shows the 72x72 region whose top-left corner sits at (144, 216); `background-size` rescales the whole atlas when display size differs from stored size. Chromium additionally supports cropping a real `<img>` element, which preserves alt text, native lazy loading, and `fetchpriority`:

```
<img src="atlas.webp" style="width:72px;height:72px;
     object-view-box:xywh(144px 216px 72px 72px)">
```

Where a real image element is needed cross-browser, a fixed-size wrapper with `overflow:hidden` around a negatively-offset `<img>` gives the same window, and SVG gives it declaratively (`<svg viewBox="144 216 72 72"><image href="atlas.webp"/>`). Canvas completes the set for programmatic UIs: `ctx.drawImage(atlas, 144, 216, 72, 72, dx, dy, 72, 72)` blits any region after a single decode. In every mechanism the browser fetches and decodes the atlas once, holds one decoded copy, and paints windows into it; per-tile cost is a paint, not a decode. The experiments use `background-position`, the mechanism with universal support.

3.2 Delivery, caching, and memory

An atlas changes the unit of caching from the tile to the bundle. With standard immutable content-addressed URLs (`atlas.3fe2a1.webp`, `Cache-Control: immutable`), an unchanged atlas costs zero requests on a warm cache, and the decode-once property still applies. The cost appears on content change: editing one tile invalidates the whole bundle, so the expected re-download per deploy grows with bundle size. Splitting the collection into k chunked atlases bounds the worst-case invalidation at 1/k of the collection while retaining nearly all of the byte and request savings, since chunking adds only a few container headers and a little grid slack, which makes chunking the practical default for collections that update piecemeal. Grouping tiles by update cadence (stable icon set in one chunk, weekly seasonal art in another) further confines invalidation to the chunk that actually changed.

The second resource to budget is decoded memory: a decoded atlas occupies width x height x 4 bytes regardless of its encoded size, so a 4096x4096 atlas holds 64 MB of RGBA for a 300 KB transfer. Individual images decode lazily and can be evicted per tile; an atlas is decoded and resident as a unit while any tile is visible. Chunking bounds this cost the same way it bounds invalidation, and below-the-fold chunks combine with lazy loading so off-screen tiles cost neither bytes nor memory.

4 Methodology

4.1 Assets and packing

Two deterministic asset classes anchor the study: 521 Twemoji 72x72 flat-art tiles (alpha composited over white) and 521 photographic 224x224 thumbnails, with the photo set additionally downscaled to 112 and 56 pixels for the size sweep and synthetic icon, product-thumbnail, and avatar corpora added for the use-case tests (Section 5.3). Tiles are packed row-major into a near-square grid; a padding variant edge-replicates each tile by 8 or 16 pixels, and a vertical-strip variant packs one tile per row band. All conditions consume identical source pixels.

4.2 Codecs and matched-quality protocol

Following the study's scope, the three dominant web formats encode each condition: libjpeg-turbo JPEG, WebP (lossy and lossless), and PNG, with the lossy codecs swept over a quality ladder $q \in \{30,50,65,80,90\}$. Quality is the mean over tiles of the per-tile luma SSIM [25], each tile scored after cropping it back out of the decoded artifact so atlas border bleed is charged to the atlas; the matched target of 0.97 is therefore a mean-tile floor, and Section 5.1 reports the per-tile spread where it is load-bearing. Bytes are compared at equal quality by log-linear interpolation of each condition's rate-distortion curve at the target; the five-point ladder is coarse, so where the atlas and individual curves nearly coincide the interpolation is unstable and equal-quality byte comparison is used instead (Section 5.3). When an atlas's entire ladder exceeds the target, its cheapest measured point is used, understating the atlas's advantage, and the value is reported as a lower bound. Three invariants validate the harness: lossless conditions score SSIM exactly 1.0; an atlas of one image is byte-identical to the individual file; padding never reduces atlas bytes.

4.3 Network testbed

Byte savings are protocol-independent; timing effects are not. The network testbed serves every condition from a Caddy server inside WSL2 with three protocol endpoints (HTTP/1.1, HTTP/2, HTTP/3 on QUIC), shapes real packets with `tc netem` (delay, bandwidth, random loss applied on the server

egress), and drives a fresh cold-cache Chromium instance per page load via Playwright. Each page stamps a timestamp after every tile is decoded and two animation frames have painted, giving a single time-to-all-tiles-visible endpoint, and records per-resource transfer sizes and the negotiated protocol from the Resource Timing API; every load is validated against the intended protocol and against the manifest's byte totals, so a page that fetched the wrong way cannot enter the dataset. Four serving conditions run: N individual files, one atlas, four chunked atlases, and a byte-bundle (the N encoded files concatenated into one binary resource plus an offset index; the client slices the buffer and decodes each tile from its own bytes, retaining per-file codec adaptation while collapsing N requests into one). The sweep crosses these with $N \in \{50, 200, 500\}$, both asset classes as WebP q80, three protocols, and four network profiles. Within each profile the load order is randomized across conditions, protocols, and repetitions, and the first repetition of each cell is discarded as a warm-up, leaving 11 measured loads per cell.

4.4 Artifact availability

All code, asset manifests, raw per-run measurements, and the construction heuristic are released at github.com/ApartsinProjects/ImageBundling, and every table and figure in this report is regenerated from that data by a single build command. The measurements use Pillow 12.2 (libwebp 1.6.0, libjpeg-turbo via libjpeg 8.0, zlib-ng 1.3.1), Python 3.14, zstandard and Brotli for the delta-update study, Caddy 2.11.4 for the three-protocol server, and Playwright 1.58 driving Chromium 145 for the network loads. The photographic corpus is drawn from Lorem Picsum and the flat-art corpus from the Twemoji set; both frozen manifests are in the repository so every condition consumes identical source pixels.

5 Results

5.1 Byte savings at matched quality

Figure 1. A 100-tile atlas (10x10 grid of 72x72px tiles, 720x720px). Encoded with identical encoder settings both ways (distinct from Table 1's matched-quality protocol), the 100 tiles cost 147,361 bytes as separate JPEG files and 112,059 bytes as this single JPEG (24% less), while 100 HTTP requests become one. Each tile is displayed with CSS `background-position`.

Table 1 reports the saving from atlasing at SSIM 0.97 across both classes. Three regularities organize the table. First, savings scale with the ratio of per-file structural cost to content bytes: 72-pixel flat-art tiles encode to 1–3 KB, so JPEG's roughly 600 bytes of per-file tables and headers alone account for most of its measured 19–26% saving across N, while WebP's 30-byte floor leaves it the smallest lossy gain (8–15%). Second, savings broadly increase with N and flatten by a few hundred tiles as amortization completes (the trend is not strictly monotonic; because each N draws one deterministic subset, small non-monotonicities such as WebP flat art at 8.3/15.2/8.2% for N = 50/200/500 reflect subset composition, not a scaling law). Third, tile size, not image count, is the dominant variable, and the photo rows of Table 1 trace the full crossover: downscaling the same 500 photographs from 224 to 112 to 56 pixels moves the JPEG saving from 3.0% to 9.8% to 29.8%, and moves WebP from −8.5% through −1.4% to +15.3%, placing WebP's bundling break-even near 100-pixel tiles. Search-result and

recommendation thumbnails (48–120 px) therefore sit squarely in the paying regime for both formats, while product-grid images (200 px and up) pay only under JPEG, and only a few percent.

To confirm the crossover is not an artifact of the single deterministic subset each Table 1 cell uses, we resampled it: for each (tile size, N, codec) we drew 20 random N-tile subsets from the full pool and computed the matched-quality saving of each. The medians track Table 1 and the 95% bootstrap intervals are tight and separate the regimes. For photos the JPEG saving is 29.1% (95% CI [28.4, 29.5]) at 56 px, 9.3% [8.9, 9.8] at 112 px, and 2.6% [2.4, 2.8] at 224 px, all clearly positive; WebP is 14.8% [10.0, 17.4] at 56 px, straddles zero at 112 px ([−1.6, 1.9]), and is clearly negative at 224 px (−8.0% [−12.0, −6.7]). The WebP break-even near 100 px is thus the point where its interval crosses zero, not a single-sample coincidence.

Table 1. Bytes saved by atlasing (%) at matched per-tile SSIM 0.97 (lossy codecs; interpolated on the quality ladder) or exact lossless comparison (-ll columns), grid packing, no padding. Negative values (red) mean the atlas is larger; n/a marks a cell whose quality ladder lies entirely above the target with no interpolable point.

class	N	JPEG	WebP	PNG	WebP-lossless
flat art 72px	10	19.4	n/a	4.3	-0.2
flat art 72px	50	26.3	8.3	5.4	5.4
flat art 72px	200	25.1	15.2	-3.1	-4.7
flat art 72px	500	26.3	8.2	-5.7	-3.6
photos 56px	10	22.3	15.3	-1.4	2.9
photos 56px	50	27.7	14.2	1.1	2.5
photos 56px	200	29.3	18.8	0.4	3.7
photos 56px	500	29.8	15.3	0.8	4.9
photos 112px	10	6.3	2.7	-3.8	1.8
photos 112px	50	8.4	1.7	-0.9	4.3
photos 112px	200	9.4	-0.1	-1.8	4.3
photos 112px	500	9.8	-1.4	-1.6	1.4
photos 224px	10	1.0	-4.3	-5.7	-3.4
photos 224px	50	1.8	-5.1	-2.8	3.0
photos 224px	200	2.6	-8.2	-3.8	0.3
photos 224px	500	3.0	-8.5	-3.6	-4.9